WebAnything: From the Wild Web to Scalable Agent Environments via Fully Automated Reconstruction

Anonymous ACL submission

Abstract

Web agents require diverse, executable, and resettable website environments for reproducible training and evaluation. Human-assisted mirrors demand per-site engineering, while generative construction can diverge from specified real websites. Capture-and-replay preserves observations but restricts interaction to recorded paths and states.

We present WEBANYTHING, a fully automated framework that reconstructs source-grounded, reusable Web environments from real URLs without per-site manual implementation or repair after initial setup and authorization. An Explorer–Builder collects browser-observable structures, resources, and interaction outcomes; an independent verifier checks frozen candidates and guides local repair or targeted source recapture.

We construct 244 website artifacts across 14 application domains and 53 categories, supporting 12,984 executed and accepted tasks. Compared with human-assisted WebHarbor, WEBANYTHING achieves higher aggregate scores on three fidelity metrics, with 12 wins, two ties, and no losses in a blind audit. Fine-tuning Qwen3.5-9B Instruct on 5,856 interaction trajectories improves REAL success by 9.9 percentage points, Odysseys-Easy Rubric Avg and Task Avg by 6.25 and 6.32 percentage points, and WebArena-Verified Overall success by 13.33 percentage points under our harness.

1 Introduction

Visual Web agents need stable, controllable environments in which they can explore, change state, recover from errors, and receive execution feedback. WebArena and VisualWebArena demonstrate the value of self-hosted websites for reproducible evaluation (Zhou et al., 2023; Koh et al., 2024); WebGym and WebFactory further study scaling environments and interaction experience (Bai et al., 2026; Fan et al., 2026). The real Web offers diverse structures and workflows, but live execution complicates state initialization, reproducibility, and safe isolation.

We investigate how specified real websites can be automatically transformed at scale into source-grounded, executable, and reusable Web agent environments, without requiring manual site-by-site reconstruction.

Existing approaches trade off **automation, source fidelity, and interaction coverage** (Figure 1). Human-assisted mirrors provide controllable source-corresponding environments, but require per-site implementation and debugging (Aiming Lab, 2026). Generative methods rapidly construct executable sites from specifications or visual references, whose limited detail may leave behavior inconsistent with a specified source (Fan et al., 2026; Zhang et al., 2026). Capture-and-replay preserves observed content but supports paths and states within recorded coverage (Shi et al., 2017; Srivastava et al., 2025); Weblica combines replay with source-grounded synthesis (Kar et al., 2026). Scaling real-website environments therefore requires both correspondence to a chosen source and flexible, repeatable interaction.

Two technical challenges arise. The **Source-Grounding Gap** concerns reconstructing structures, resources, and behaviors when private backends and website internals are inaccessible. Screenshots and model priors alone can omit state relationships. Browser observations must instead be organized into evidence that constrains implementation. The **Verification and Feedback Gap** concerns detecting and correcting reconstruction discrepancies. Builder self-checks can overlook source-inconsistent defects, and a discrepancy may reflect either faulty implementation or insufficient observation. These cases require different actions: local repair or additional source capture.

We present WEBANYTHING, a framework for

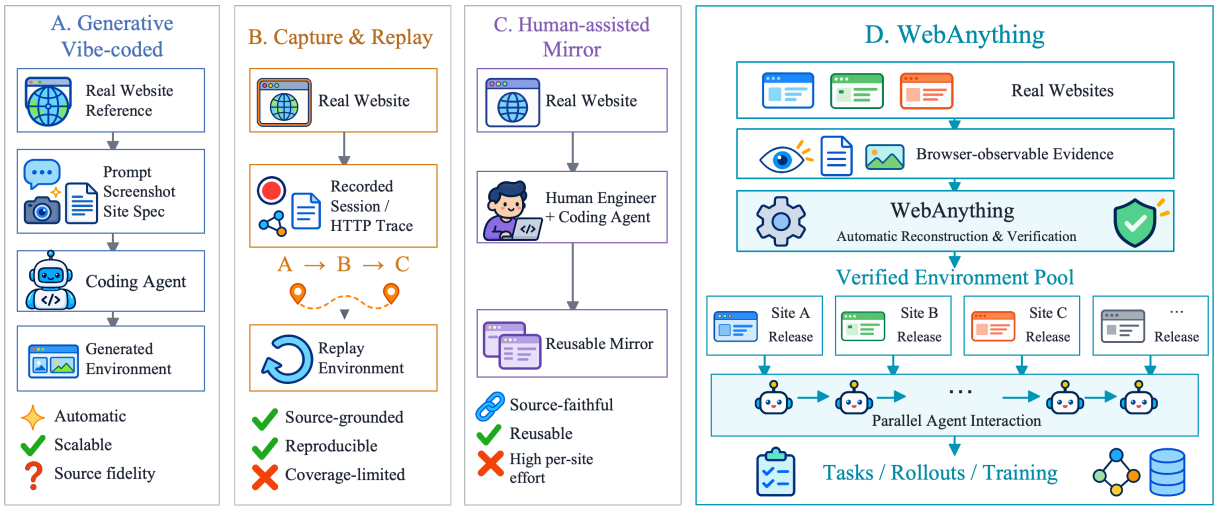


Figure 1: Trade-offs in real-website environment construction. Human-assisted mirrors require per-site engineering; generative methods depend on limited references for source fidelity; replay restricts interaction to captured paths. WEBANYTHING combines source-grounded reconstruction with independent verification and iterative refinement.

scalable, source-grounded Web agent environment construction. Starting from real URLs, a coupled Explorer–Builder captures page structures, resources, and before/after interaction observations, then implements executable local routes and state transitions. An Independent Verifier checks frozen candidates in a separate context against source evidence and site contracts. Evidence-supported implementation errors trigger local repair; unresolved evidence gaps trigger targeted recapture. Updated candidates are frozen and reverified before release within their declared scope. Fixed initial states, reset mechanisms, and task evaluators support repeated interaction and trajectory collection.

Our evaluation covers construction scale, environment quality, and downstream utility. The resource pool contains **244 website artifacts** across 14 domains and 53 categories. On five matched WebHarbor sites and 14 historical states, WEBANYTHING achieves higher automated fidelity metrics and 12 wins, two ties, and no losses in a blind overall-fidelity audit. Separate interaction tests characterize functional coverage. Historical analysis supports self-check omissions in **39 pairs across 22 sites** and independently reverified resolution of 18 issue records across six sites. The environments support **12,984 executed and accepted tasks**; SFT on earlier interaction trajectories improves Qwen3.5-9B Instruct on REAL and Odysseys-Easy.

Our contributions are:

- **Fully automated, source-grounded reconstruction.** We introduce a unified Explorer–Builder that automatically reconstructs executable Web environments from real URLs, using browser-observable structures, resources, and behaviors without per-site manual implementation or debugging.
- **Independent verification and evidence-aware refinement.** An independent verifier checks frozen candidates against source evidence and site contracts, directing discrepancies to local repair or targeted recapture, followed by automated revision and reverification.
- **Scalable environment and task supply.** We construct 244 website artifacts across 14 domains, supporting 12,984 executed and accepted tasks. Historical construction records, independent quality evaluations, and downstream SFT demonstrate automation, source fidelity, and practical utility.

2 Related Work

Real-Web tasks and executable environments. Mind2Web, WebVoyager, WebGym, WebLINX, SeeAct, Online-Mind2Web, and AssistantBench study tasks or interactions on real websites (Deng et al., 2023; He et al., 2024; Bai et al., 2026; Lù et al., 2024; Zheng et al., 2024a; Xue et al., 2025; Yoran et al., 2024). WebArena, VisualWebArena, REAL, and WebShop emphasize controlled execution and state initialization (Zhou et al., 2023; Koh et al., 2024; Garg et al., 2025; Yao et al., 2022). WorkArena, WorkArena++, and BrowserGym support enterprise workflows and common evaluation

infrastructure, while OSWorld and AgentStudio extend executable evaluation to computer environments (Drouin et al., 2024; Boisvert et al., 2024; Le Sellier De Chezelles et al., 2025; Xie et al., 2024; Zheng et al., 2024b). WEBANYTHING connects real-source diversity with repeatable execution by constructing local environments for specified websites.

Website generation, replay, and mirroring. WebFactory and InfiniteWeb scale executable sites from specifications and references; VeriEnv and WebArena-Infinity incorporate cloning or iterative testing (Fan et al., 2026; Zhang et al., 2026; Chae et al., 2026a,b; WebArena-Infinity Contributors, 2026). WebGen-Agent combines VLM appearance feedback and GUI testing with backtracking and best-candidate selection for instruction-driven website generation (Lu et al., 2026). Our refinement instead anchors frozen candidates to a specified source URL and routes discrepancies according to source-evidence sufficiency. Screenshot-to-code methods include pix2code, Pix2Struct, WebSight, and Design2Code (Beltramelli, 2017; Lee et al., 2023; Laurençon et al., 2024; Si et al., 2025). World of Bits and WARC-Bench preserve captured content through replay (Shi et al., 2017; Srivastava et al., 2025); Weblica combines replay with source-grounded synthesis (Kar et al., 2026). WebHarbor constructs mirrors through human–Coding Agent collaboration (Aiming Lab, 2026). WEBANYTHING jointly uses structural, resource, and behavioral evidence. Separate visual and interaction measurements complement image metrics such as SSIM and LPIPS (Wang et al., 2004; Zhang et al., 2018).

Feedback and interaction-data supply. Self-Refine and Reflexion use feedback for output revision and agent reflection (Madaan et al., 2023; Shinn et al., 2023). Our feedback targets frozen website implementations and missing source evidence. AutoGUIWorld synthesizes GUI observations and trajectories, while WEFT studies coordination among environments, tasks, harnesses, and evaluators (Yang et al., 2026; Mao et al., 2026). Synatra and AgentTrek derive demonstrations from indirect knowledge or tutorials; WebRL studies self-evolving task curricula (Ou et al., 2024; Xu et al., 2024; Qi et al., 2024). OS-Atlas, UI-TARS, and OpenCUA expand GUI grounding or interaction data (Wu et al., 2024; Qin et al., 2025; Wang et al., 2025). WEBANYTHING supplies executable environments and collects interaction data with a fixed NorthCUA harness and a unified task/evaluator pipeline, measuring utility on external benchmarks.

3 Methodology

WEBANYTHING uses source evidence as the shared basis for construction and independent verification (Figure 2). An Explorer–Builder reconstructs executable local pages and states; verification feedback then drives implementation repair or targeted source recapture.

3.1 Problem Formulation

Given a website W, entry URL, permitted access scope, and construction budget, we seek a local environment $\widehat{W}$ preserving observed presentation, interactions, and state changes within a site contract $\mathcal{C}$. The source is accessible through browser observations rather than private backend code. Its evidence is

$$\mathcal{E} = (\mathcal{E}_p, \mathcal{E}_r, \mathcal{E}_b), \quad \mathcal{E}_b = \{(o_t, a_t, o_{t+1})\}_{t=1}^{N}, \quad (1)$$

where $\mathcal{E}_p$ records page structure and appearance, $\mathcal{E}_r$ resources and dependencies, and $\mathcal{E}_b$ observations before and after actions. The contract specifies required pages, workflows, and checkable outcomes. Reconstruction translates these observations into executable local state transitions that support different tasks within the implemented scope.

3.2 Source-Grounded Reconstruction

State-associated evidence. For each visited state, the Explorer–Builder associates page structures, computed styles, element geometry, screenshots, and resource references. Actions are linked to subsequent visible feedback and state changes. Distinct interaction states at the same URL remain distinct evidence: a closed menu and its expanded state, for example, constrain different implementation requirements.

Evidence-constrained implementation. Structural observations constrain layout and component relationships; resource observations ground fonts, images, and media; behavioral observations constrain navigation, filtering, and state modification. The Builder organizes these into local components, routes, and shared object relationships. Screenshot references thus complement structural and behavioral evidence. A specific discrepancy can be traced back to its associated page state and action.

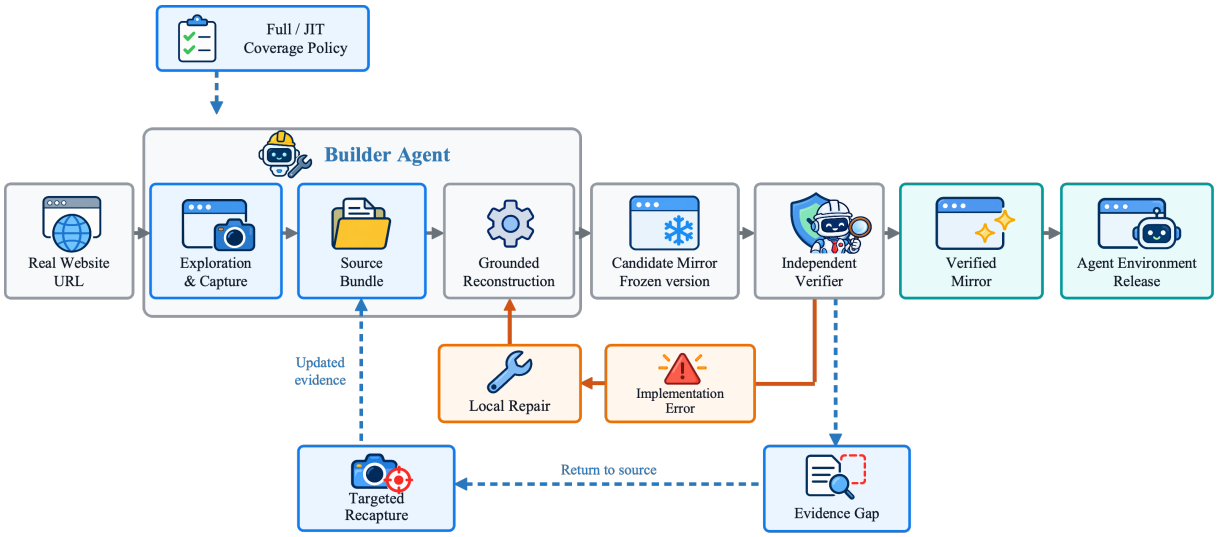


Figure 2: WEBANYTHING reconstruction pipeline. Source evidence guides executable construction. Independent checks of frozen candidates drive local repair or targeted recapture; verified candidates are released with repeatable interaction and task-evaluation interfaces.

When available observations do not establish a required outcome, the system seeks additional evidence rather than treating an inferred behavior as observed.

Coverage policies. **Full Coverage** expands accessible page types, interaction states, and neighboring branches from the entry URL and native navigation. **Just-in-Time Coverage** prioritizes dependencies of a given task set and expands them through construction and verification feedback. Both share the reconstruction and acceptance mechanisms, differing in exploration scope and expansion timing. Reported construction experiments use Full Coverage.

3.3 Independent Verification and Evidence-Aware Refinement

Verification decoupling. The Builder implements and self-checks a candidate; an Independent Verifier then examines the frozen implementation in a separate context. It obtains rendering and operation results anew and compares them with source evidence and the site contract. Applicable checks cover pages, controls, resources, workflows, persistence, and restoration. The Verifier cannot edit candidate code. Stateful tests operate through local UI/API interfaces and restore the initial state. Candidate freezing binds findings to the inspected implementation and prevents verification-time repair from changing the target.

Evidence-aware refinement. The model diagnoses each discrepancy using check results, source observations, the contract, and runtime conditions. If existing evidence establishes an expected outcome that the candidate fails to reproduce, **Local Repair** returns the affected page or control, executed action, expected and observed outcomes, and supporting evidence to the Builder. If evidence is insufficient to establish the target outcome, **Targeted Recapture** forms a concrete observation request and revisits the relevant source region. Newly captured structures, resources, or behaviors constrain the next implementation.

For example, Figma's authorized source capture established a canvas-anchored comment composer. Independent checks found mismatched placement and appearance, providing a concrete target for local revision. The Builder read the feedback, changed the editor markup and styles, and submitted a new candidate; independent checks then confirmed the composer's geometry and visibility. Existing source evidence supported repair in this case; an unresolved observation question instead becomes a bounded recapture request (Appendix B.8).

After either path updates the candidate, the controller freezes it again and the Verifier repeats affected checks and applicable regression tests. Verification therefore connects discrepancies to actionable revision, while source evidence constrains both initial reconstruction and subsequent refinement. Candidates passing checks within their de-

cleared scope can proceed to release. Implementation details and original prompts appear in Appendix B.4.

3.4 Environment and Verifiable Task Supply

Released environments package local implementations, resources, initial data, runtime configuration, and reset interfaces. Shared state connects related pages, and applicable creation, modification, and save operations affect later interactions. Fixed initial data and state restoration provide repeatable task starting conditions. The task pipeline generates instructions and matching success conditions for implemented capabilities. Agents execute tasks through the fixed NorthCUA harness, and evaluators check final answers or terminal environment states. Accepted trajectories are converted into step-level training examples. Task generation, executed acceptance, trajectory collection, and training export remain separate stages and counting units; Section 4.4 reports their outputs.

4 Experiments and Results

We study **RQ1: construction scale and efficiency**, **RQ2: environment quality and independent verification**, and **RQ3: task support and downstream learning utility**.

4.1 Experimental Setup

Objects and baselines. The resource inventory covers 244 unique website artifacts as of October 9, 2026. Full Coverage construction records for 85 sites support automation and expenditure analysis. Quality comparisons use frozen artifacts and an independent evaluation pipeline: five WebHarbor sites with 14 historical reference states, and five VeriEnv artifacts for offline interaction tests. Historical VeriEnv fidelity references cover four LinkedIn states. Subsets and reference conditions are specified in Appendices A.2–A.4.

Measurements. Pixel Similarity, SSIM, and Text F1 measure source-state correspondence, aggregated within sites and then averaged equally across sites. A blind audit of the 14 WebHarbor-matched states uses final labels agreed through discussion by three annotators, with randomized anonymous mirror order. Interaction tests disable external network access and report capability coverage, executed-test success, and all-test pass, keeping their denominators distinct. Explicit adapters restore initial states. Historical verification analysis links self-checks, independent findings, and repair traces; its definitions appear in Appendix B.7.

Tasks and training. Two disjoint collections contain 6,172 accepted tasks across 74 sites and 6,812 across 229 sites. SFT uses earlier WebAnything interaction data: 5,856 trajectories associated with 5,167 unique training task IDs, converted into 60,517 step-level examples. Multiple trajectories or format variants can share a task ID. The newer 6,812 tasks are not used in this SFT experiment. We fine-tune Qwen3.5-9B Instruct for 473 steps (the fixed training endpoint), with global batch size 256 and learning rate 5×10^{-6}. Evaluation uses this trained model on REAL (Garg et al., 2025), Odysseys-Easy (Jang et al., 2026), and WebArena-Verified (El hattami et al., 2025); configurations and overlap checks appear in Appendix E.1.

4.2 RQ1: Construction Scale and Efficiency

Cross-domain supply. The unified pipeline produces **244 website artifacts** across **14 application domains and 53 categories** (Figure 3), spanning diverse layouts, navigation structures, and workflows. Artifacts are counted by site identity and recorded homepage availability; deeper fidelity and functionality are assessed separately on evaluated subsets. Appendix F.1 lists the sites, and Appendix A.6 reports additional rendering diagnostics.

Automation and recorded expenditure. After initial configuration and necessary authorization, the 85-site historical collection contains no recorded per-site manual implementation, debugging, or repair. Exploration, capture, implementation, and revisions are performed by the pipeline. Recoverable, priceable model-call expenditure for 84 sites totals **$220.62**, with mean **$2.63** and median **$1.60** per recorded site. These partial model-call costs exclude unpriced calls and system overhead; historical outcomes and accounting details are in Appendix A.1.

4.3 RQ2: Environment Quality and Independent Verification

A. Source-state fidelity. On five matched WebHarbor sites and 14 historical states, WEBANYTHING scores higher on all three aggregate metrics: Pixel Similarity **0.9240 vs. 0.8083**, SSIM **0.7904 vs. 0.6260**, and Text F1 **0.8541 vs. 0.2617** (Table 1a). The blind overall-fidelity audit gives **12 wins, two**

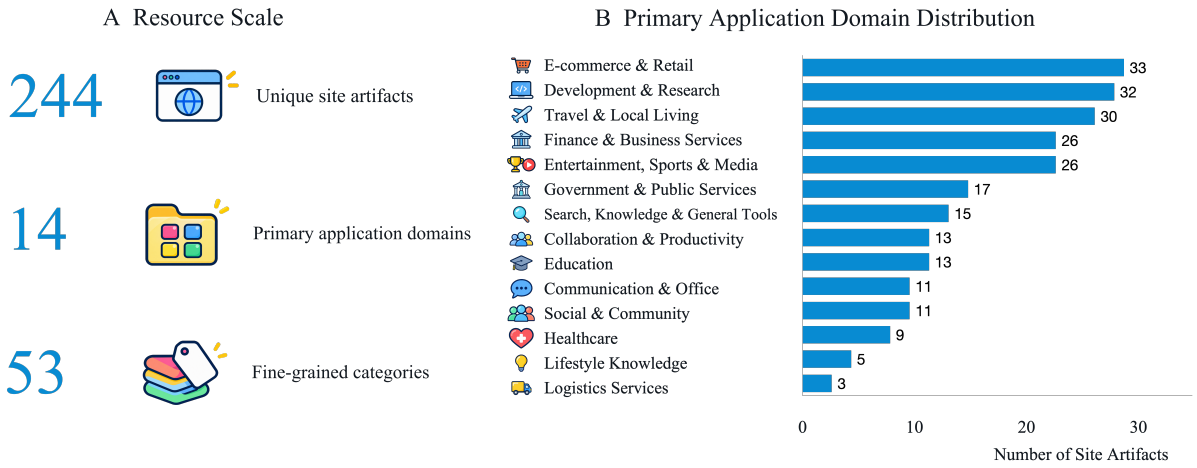


Figure 3: Cross-domain resource supply: 244 website artifacts across 14 application domains and 53 fine-grained categories, with the domain distribution at right.

ties, and no losses; separate information-fidelity and discrepancy-severity judgments show complementary differences. Human judgments concern visible reference states, while functionality is tested through execution. The four-state LinkedIn comparison also favors WEBANYTHING in Pixel and SSIM, reported separately.

Historical references include rendering defects, and source/mirror JavaScript conditions differ for Text F1. Appendix A.3 specifies these conditions and sensitivity results; Appendix D gives the blind-audit protocol. Selected page comparisons appear in Figure 4 in the appendix.

B. Interaction validity. Offline tests cover navigation, retrieval, forms, state mutations, and applicable recovery operations (Table 1b). On the WebHarbor subset, WEBANYTHING achieves 75% capability coverage and 63.3% all-test pass, versus WebHarbor's 80% and 76.7%, showing broader tested interaction coverage for WebHarbor. On the separate VeriEnv subset, selected pool artifacts pass **29/33 tests**, versus **21/33** for VeriEnv; results vary by site, with VeriEnv ahead on LinkedIn. Artifact inclusion and per-site results appear in Appendix A.7.

Together, the comparisons show that *source fidelity determines what agents observe, while interaction validity determines what they can execute*. Higher visual correspondence alone does not establish broader functional coverage.

C. Independent verification and feedback. Among 229 linked historical report pairs across 69 websites, 219 follow passing Builder self-checks. Independent verification records 56 passes, 139 failures, and 24 source-blocked outcomes (Table 6 in the appendix). Inspection supports concrete self-check omissions in **39 pairs across 22 sites**, concerning interactions, appearance, state management, resources, and workflows. In Stashmedia, independent checks find external images and broken assets in the same frozen candidate whose resource self-checks passed.

A separate repair-trace cohort contains feedback, revision, and subsequent verification records for 46 sites. Issue-level inspection confirms **18 independently reverified resolution records across six sites**: 14 local implementation repairs, three runtime repairs, and one request-lifecycle issue. Ctrip illustrates five resolved conditions, including login rejection, destination search, favorites, booking drafts, and reset (Figure 5 in the appendix).

These observations support additional defect discovery and traceable refinement; heterogeneous historical pairing and incomplete issue adjudication preclude interpreting them as causal gains or overall omission/repair rates. Appendix B.7 documents evidence levels, repeated issue manifestations, and the inspected cohort.

4.4 RQ3: Task Support and Downstream Learning Utility

A. Executed task supply. Agents and evaluators yield **12,984 executed and accepted tasks** from two disjoint collections. The newer batch supplies **6,812 tasks across 229 sites**: 217 reach 30 tasks, 12 partially complete the quota, and 15 of the 244 pool sites have none in this batch. Completion relative to the $244 \times 30 = 7{,}320$ target is **93.1%**. The earlier collection supplies 6,172 tasks across 74

A: Automated fidelity metrics					
Comparison subset	System	Sites/states	Pixel ↑	SSIM ↑	Text F1 ↑
Five shared WebHarbor sites	WEBANYTHING	5 / 14	0.9240	0.7904	0.8541
Same subset	WebHarbor	5 / 14	0.8083	0.6260	0.2617
VeriEnv LinkedIn	WEBANYTHING	1 / 4	0.9574	0.8190	N/A
Same subset	VeriEnv	1 / 4	0.9436	0.7637	N/A

B: Human blind audit				
Assessment	Wins	Ties	Losses	Win rate ↑
Overall fidelity (Q1)	12	2	0	85.7%
Information fidelity (Q2)	10	3	1	71.4%
System (Q3)	Equivalent	Minor	Severe	Mean severity ↓
WEBANYTHING	12/14	2/14	0/14	0.143
WebHarbor	0/14	8/14	6/14	1.429

Q3 is coded as 0 (equivalent), 1 (minor differences that do not affect functionality), and 2 (severe discrepancies potentially affecting an agent). Human judgments assess visible historical states.

Table 1a: Historical source-state fidelity. Automated metrics are aggregated within sites, then equally averaged across the five-site, 14-state WebHarbor subset; the four historical LinkedIn states are reported separately. Human blind-audit wins, ties, and losses compare WEBANYTHING against WebHarbor.

System	Capability coverage	Executed-test success	All-test pass
Five-site WebHarbor comparison			
WEBANYTHING	15/20 (75%)	19/20 (95%)	19/30 (63.3%)
WebHarbor	16/20 (80%)	23/23 (100%)	23/30 (76.7%)
Five-site VeriEnv comparison			
Selected pool artifacts	22/22 (100%)	29/31 (93.5%)	29/33 (87.9%)
VeriEnv	20/22 (90.9%)	21/28 (75.0%)	21/33 (63.6%)

Table 1b: Offline interaction validity on separate five-site subsets. Capability coverage counts tested capabilities; executed-test success uses executed tests; all-test pass retains unsupported and blocked tests. Persistence and state-restoration results appear in Appendix A.7.

sites, whose sites are a subset of the newer batch's sites. Acceptance definitions and snapshot statistics are in Appendix C.3.

B. Diversity and complexity. The earlier tasks cover retrieval, search/filtering, comparison, and state operations. Automated, overlapping capability labels mark 26.8% for state operations and 37.7% for cross-site attributes. Observed tool-interaction depth averages **10.4 steps** (median 8, 90th percentile 21, maximum 89); **34.7%** exceed 10 steps. These executions demonstrate support for queries and multi-step workflows. Figure 6 and Appendix C.2 provide full distributions and label definitions.

C. Downstream learning. SFT on the 5,856 WebAnything interaction trajectories improves performance on three external benchmarks (Table 2). On 91 jointly valid REAL tasks, success increases from **22/91 to 31/91** (+9.9 percentage points). Retaining infrastructure failures in the full 102-task set gives 21.6% vs. 30.4% (+8.8pp). On 44 jointly valid Odysseys-Easy tasks, Rubric Avg rises from **19.71% to 25.96%** (+6.25pp), Task Avg from **18.86% to 25.18%** (+6.32pp), and fully successful tasks from 2/44 to 4/44.

On WebArena-Verified, our internal harness reports an Overall gain of **13.33pp** (16.41% to 29.74%), with substantial gains on Navigate (+18.49pp) and Mutate (+31.50pp). Retrieve decreases by 8.75pp under strict scoring. Supplementary rescoring of the same SFT outputs, ignoring JSON-format judgment, yields 105/325 passes (32.31%) versus 39/325 (12.00%): 66 additional passes. This reveals format-sensitive failures while retaining the strict scores as the main results. The scoring change is not a training gain; the baseline's format-relaxed result is unavailable.

The confirmed REAL and Odysseys-Easy gains support the learning utility of environment-

Training setting	REAL	Odysseys-Easy		
	Success ↑	Rubric Avg ↑	Task Avg ↑	Perfect ↑
9B Instruct	24.2	19.71	18.86	4.55
+ WebAnything interaction data	34.1	25.96	25.18	9.09
Δ	+9.9pp	+6.25pp	+6.32pp	+4.55pp

WebArena-Verified: internal harness

Task type	9B Instruct	WebAnything SFT	Δ
Retrieve	20.75	12.00	−8.75pp
Navigate	22.22	40.71	+18.49pp
Mutate	10.51	42.01	+31.50pp
Overall	16.41	29.74	+13.33pp

Δ denotes percentage points. Perfect counts are 2/44 and 4/44. **Same-model Retrieve rescoring:** ignoring JSON-format judgment increases SFT passes from 39/325 (12.00%, strict) to 105/325 (32.31%), or 66 additional passes (+20.31pp). This supplement changes scoring, not the model or agent execution.

Table 2: Downstream SFT results. REAL and Odysseys-Easy use 91 and 44 jointly valid tasks. WebArena-Verified uses our internal harness with strict scoring. Values are percentages.

interaction data. The before/after comparison measures SFT transfer, rather than isolating individual reconstruction mechanisms.

5 Conclusion

We present WEBANYTHING, an end-to-end automated framework for transforming real websites into source-grounded, executable, and reusable Web agent environments. Unlike human-assisted mirroring approaches, WEBANYTHING automates source exploration, evidence collection, website reconstruction, independent verification, and iterative refinement, eliminating the need for per-site manual implementation and debugging in our audited construction runs. By grounding reconstruction in browser-observable structures, resources, and interaction behaviors, the framework reduces reliance on incomplete visual references or manually specified workflows.

Structural, resource, and behavioral evidence guides reconstruction, while independent verification directs discrepancies to local repair or targeted recapture. This separation enables the system to distinguish implementation defects from insufficient source evidence and refine reconstructed environments accordingly. The framework constructs 244 website artifacts across 14 application domains and 53 categories, supporting 12,984 executed and accepted tasks. Experiments demonstrate source fidelity, operational validity, and verification-driven refinement, while also highlighting the importance of evaluating functional coverage independently from visual similarity. Furthermore, supervised fine-tuning on 5,856 interaction trajectories improves Web agent performance on REAL, Odysseys-Easy, and WebArena-Verified, demonstrating the downstream utility of the constructed environments.

Together, these results establish a practical path toward scaling real-Web agent environments with minimal site-specific human engineering. More broadly, WEBANYTHING connects automated website reconstruction with reproducible interaction and training-data collection, providing a foundation for scalable Web agent development and post-training.

Limitations

Training validation with large-scale models. Our supervised fine-tuning experiments with Qwen3.5-9B Instruct demonstrate the learning value of interaction data from reconstructed environments for downstream Web agent tasks. However, current training experiments primarily concern the 9B parameter scale. We have not systematically evaluated these data with larger foundation models, different post-training methods, or larger training budgets. Future work can examine relationships among environment scale, data diversity, and model capabilities to assess this approach in larger-scale agent training.

Continuous real-Web updates and environment maintenance. Real websites continually change their page structures, resources, and interaction behaviors. WEBANYTHING currently reconstructs and verifies source states at particular times, without systematically evaluating the timeliness of local mirrors or long-term maintenance efficiency under continuing source changes. Future research can investigate source-change detection, incremental evidence capture, and local environment updates to reduce maintenance costs while preserving source correspondence.

Copyright, content authorization, and environment distribution. High-fidelity reconstruction of real websites involves page content, images, fonts, and branding that may be protected by copyright, trademarks, and terms of service. Public accessibility does not necessarily grant permission to reproduce, modify, or redistribute content. Environment construction, research use, and public release must therefore comply with applicable content authorization and compliance requirements. Future releases can prioritize the general construction framework, evaluation protocols, and de-identified research data, while determining the distribution scope of website mirrors and original resources according to their licenses.

Ethical Considerations

Exploring, capturing, and reconstructing real websites raises issues concerning terms of service, intellectual property, privacy, and security. The system should process only websites covered by authorization, public licenses, or research agreements, avoid collecting account credentials, cookies, personal information, and payment content, and run released environments in isolated networks and sandboxes. For websites or tasks that could facilitate abusive automation, reconstruction scope, public content, and model capabilities should be restricted.

References

Aiming Lab. 2026. Webharbor: Docking real websites for evolving gui agent environments. https://github.com/aiming-lab/WebHarbor. Open-source project; accessed 2026-10-06.

Hao Bai, Alexey Taymanov, Tong Zhang, Aviral Kumar, and Spencer Whitehead. 2026. Webgym: Scaling training environments for visual web agents with realistic tasks. *arXiv preprint arXiv:2601.02439*.

Tony Beltramelli. 2017. pix2code: Generating code from a graphical user interface screenshot. *arXiv preprint arXiv:1705.07962*.

Léo Boisvert, Megh Thakkar, Maxime Gasse, Massimo Caccia, Thibault Le Sellier De Chezelles, Quentin Cappart, Nicolas Chapados, Alexandre Lacoste, and Alexandre Drouin. 2024. WorkArena++: Towards compositional planning and reasoning-based common knowledge work tasks. *arXiv preprint arXiv:2407.05291*.

Hyungjoo Chae, Jungsoo Park, and Alan Ritter. 2026a. Safe and scalable web agent learning via recreated websites. *arXiv preprint arXiv:2603.10505*.

Hyungjoo Chae, Jungsoo Park, and Alan Ritter. 2026b. VeriEnv: Official repository. https://github.com/kyle8581/VeriEnv. Architecture and reset documentation; accessed 2026-10-06.

Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Sam Stevens, Boshi Wang, Huan Sun, and Yu Su. 2023. Mind2Web: Towards a generalist agent for the web. In *Advances in Neural Information Processing Systems*, volume 36.

Alexandre Drouin, Maxime Gasse, Massimo Caccia, Issam H. Laradji, Manuel Del Verme, Tom Marty, David Vazquez, Nicolas Chapados, and Alexandre Lacoste. 2024. WorkArena: How capable are web agents at solving common knowledge work tasks? In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 11642–11662.

Amine El hattami, Megh Thakkar, Nicolas Chapados, and Christopher Pal. 2025. WebArena Verified: Reliable evaluation for web agents. In *Workshop on Scaling Environments for Agents*.

Sicheng Fan, Qingyun Shi, Shengze Xu, Shengbo Cai, Tieyong Zeng, Li Ling, Yanyi Shang, and Dehan Kong. 2026. Webfactory: Automated compression of foundational language intelligence into grounded web agents. *arXiv preprint arXiv:2603.05044*.

Divyansh Garg, Shaun VanWeelden, Diego Caples, Andis Draguns, Nikil Ravi, Pranav Putta, Naman Garg, Tomas Abraham, Michael Lara, Federico Lopez, James Liu, Atharva Gundawar, Prannay Hebbar, Youngchul Joo, Charles London, Christian Schroeder de Witt, and Sumeet Motwani. 2025. REAL: Benchmarking autonomous agents on deterministic simulations of real websites. *arXiv preprint arXiv:2504.11543*.

Hongliang He, Wenlin Yao, Kaixin Ma, Wenhao Yu, Yong Dai, Hongming Zhang, Zhenzhong Lan, and Dong Yu. 2024. WebVoyager: Building an end-to-end web agent with large multimodal models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, pages 6864–6890.

Lawrence Keunho Jang, Jing Yu Koh, Daniel Fried, and Ruslan Salakhutdinov. 2026. Odysseys: Benchmarking web agents on realistic long horizon tasks. *arXiv preprint arXiv:2604.24964*.

Oğuzhan Fatih Kar, Roman Bachmann, Yuanzheng Gong, Anders Boesen Lindbo Larsen, and Afshin Dehghan. 2026. Weblica: Scalable and reproducible training environments for visual web agents. *arXiv preprint arXiv:2605.06761*.

Jing Yu Koh, Robert Lo, Lawrence Jang, Vikram Duvvur, Ming Chong Lim, Po-Yu Huang, Graham Neubig, Shuyan Zhou, Ruslan Salakhutdinov, and Daniel Fried. 2024. Visualwebarena: Evaluating multimodal agents on realistic visual web tasks. *arXiv preprint arXiv:2401.13649*.

Hugo Laurençon, Léo Tronchon, and Victor Sanh. 2024. Unlocking the conversion of web screenshots into HTML code with the WebSight dataset. *arXiv preprint arXiv:2403.09029*.

Thibault Le Sellier De Chezelles, Maxime Gasse, Alexandre Drouin, Massimo Caccia, Léo Boisvert, Megh Thakkar, Tom Marty, Rim Assouel, Sahar Omidi Shayegan, Lawrence Keunho Jang, Xing Han Lù, Ori Yoran, Dehan Kong, Frank F. Xu, Siva Reddy, Quentin Cappart, Graham Neubig, Ruslan Salakhutdinov, Nicolas Chapados, and Alexandre Lacoste. 2025. The BrowserGym ecosystem for web agent research. *Transactions on Machine Learning Research*.

Kenton Lee, Mandar Joshi, Iulia Raluca Turc, Hexiang Hu, Fangyu Liu, Julian Martin Eisenschlos, Urvashi Khandelwal, Peter Shaw, Ming-Wei Chang, and Kristina Toutanova. 2023. Pix2Struct: Screenshot parsing as pretraining for visual language understanding. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 18893–18912.

Xing Han Lù, Zdeněk Kasner, and Siva Reddy. 2024. WebLINX: Real-world website navigation with multi-turn dialogue. *arXiv preprint arXiv:2402.05930*.

Zimu Lu, Houxing Ren, Yunqiao Yang, Ke Wang, Zhuofan Zong, Junting Pan, Mingjie Zhan, and Hongsheng Li. 2026. WebGen-Agent: Enhancing interactive website generation with multi-level feedback and step-level reinforcement learning. In *International Conference on Learning Representations*.

Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. 2023. Self-Refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems*, volume 36.

Bo Mao, Hang He, Linting Wang, Lizhi Lin, Maosen Zhou, Guanming Liu, Jinxiu Liu, Tianyu Huai, Chaoyun Zhang, Bingxuan Li, Kepeng Lei, Guanting Dong, Zhou Shao, Rui Zheng, Hang Yan, Jie Zhou, Chengcheng Wan, Tao Gui, Liang He, and Xipeng Qiu. 2026. WEFT: Scaling tool-use post-training for general-purpose agents. *arXiv preprint arXiv:2609.36887*.

Tianyue Ou, Frank F. Xu, Aman Madaan, Jiarui Liu, Robert Lo, Abishek Sridhar, Sudipta Sengupta, Dan Roth, Graham Neubig, and Shuyan Zhou. 2024. Synatra: Turning indirect knowledge into direct demonstrations for digital agents at scale. *arXiv preprint arXiv:2409.15637*.

Zehan Qi, Xiao Liu, Iat Long Iong, Hanyu Lai, Xueqiao Sun, Wenyi Zhao, Yu Yang, Xinyue Yang, Jiadai Sun, Shuntian Yao, Tianjie Zhang, Wei Xu, Jie Tang, and Yuxiao Dong. 2024. WebRL: Training LLM web agents via self-evolving online curriculum reinforcement learning. *arXiv preprint arXiv:2411.02337*.

Yujia Qin, Yining Ye, Junjie Fang, Haoming Wang, Shihao Liang, Shizuo Tian, Junda Zhang, Jiahao Li, Yunxin Li, Shijue Huang, Wanjun Zhong, Kuanye Li, Jiale Yang, Yu Miao, Woyu Lin, Longxiang Liu, Xu Jiang, Qianli Ma, Jingyu Li, and 16 others. 2025. UI-TARS: Pioneering automated GUI interaction with native agents. *arXiv preprint arXiv:2501.12326*.

Tianlin Shi, Andrej Karpathy, Linxi Fan, Jonathan Hernandez, and Percy Liang. 2017. World of bits: An open-domain platform for web-based agents. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 3135–3144.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 36.

Chenglei Si, Yanzhe Zhang, Ryan Li, Zhengyuan Yang, Ruibo Liu, and Diyi Yang. 2025. Design2Code: Benchmarking multimodal code generation for automated front-end engineering. In *Proceedings of the 2025 Conference of the Nations of the Americas*

Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers), pages 3956–3974.

Sanjari Srivastava, Gang Li, Cheng Chang, Rishu Garg, Manpreet Kaur, Charlene Y. Lee, Yuezhang Li, Yining Mao, Ignacio Cases, Yanan Xie, and Peng Qi. 2025. WARC-Bench: Web archive based benchmark for gui subtask executions. *arXiv preprint arXiv:2510.09872*.

Xinyuan Wang, Bowen Wang, Dunjie Lu, Junlin Yang, Tianbao Xie, Junli Wang, Jiaqi Deng, Xiaole Guo, Yiheng Xu, Chen Henry Wu, Zhennan Shen, Zhuokai Li, Ryan Li, Xiaochuan Li, Junda Chen, Boyuan Zheng, Peihang Li, Fangyu Lei, Ruisheng Cao, and 20 others. 2025. OpenCUA: Open foundations for computer-use agents. *arXiv preprint arXiv:2508.09123*.

Zhou Wang, Alan C. Bovik, Hamid R. Sheikh, and Eero P. Simoncelli. 2004. Image quality assessment: From error visibility to structural similarity. *IEEE Transactions on Image Processing*, 13(4):600–612.

WebArena-Infinity Contributors. 2026. WebArena-Infinity: Generating browser environments with verifiable tasks at scale. `https://github.com/web-arena-x/webarena-infinity`. Official repository; accessed 2026-10-06.

Zhiyong Wu, Zhenyu Wu, Fangzhi Xu, Yian Wang, Qiushi Sun, Chengyou Jia, Kanzhi Cheng, Zichen Ding, Liheng Chen, Paul Pu Liang, and Yu Qiao. 2024. OS-ATLAS: A foundation action model for generalist GUI agents. *arXiv preprint arXiv:2410.23218*.

Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. 2024. OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments. In *Advances in Neural Information Processing Systems*, volume 37.

Yiheng Xu, Dunjie Lu, Zhennan Shen, Junli Wang, Zekun Wang, Yuchen Mao, Caiming Xiong, and Tao Yu. 2024. AgentTrek: Agent trajectory synthesis via guiding replay with web tutorials. *arXiv preprint arXiv:2412.09605*.

Tianci Xue, Weijian Qi, Tianneng Shi, Chan Hee Song, Boyu Gou, Dawn Song, Huan Sun, and Yu Su. 2025. An illusion of progress? assessing the current state of web agents. In *Proceedings of the Second Conference on Language Modeling*.

Cheng Yang, Yifan Wu, Yutao Huang, Zhaohua Zhang, Beiduo Chen, Muxi Chen, Chenchen Zhao, Hexuan Deng, Haolin Yang, Geyuan Zhu, Sa Zhu, Jianhuan Zhuo, Qiuyong Xiao, Jianhao Ruan, Yiran Peng, Jiayi Zhang, Tian Ye, Xinlei Yu, Tianwen Jiang, and 2 others. 2026. AutoGUIWorld: Image generators as visual world models for GUI agent. *arXiv preprint arXiv:2610.01215*.

Shunyu Yao, Howard Chen, John Yang, and Karthik Narasimhan. 2022. WebShop: Towards scalable real-world web interaction with grounded language agents. In *Advances in Neural Information Processing Systems*, volume 35.

Ori Yoran, Samuel Joseph Amouyal, Chaitanya Malaviya, Ben Bogin, Ofir Press, and Jonathan Berant. 2024. AssistantBench: Can web agents solve realistic and time-consuming tasks? *arXiv preprint arXiv:2407.15711*.

Richard Zhang, Phillip Isola, Alexei A. Efros, Eli Shechtman, and Oliver Wang. 2018. The unreasonable effectiveness of deep features as a perceptual metric. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 586–595.

Ziyun Zhang, Zezhou Wang, Xiaoyi Zhang, Zongyu Guo, Jiahao Li, Bin Li, and Yan Lu. 2026. Infiniteweb: Scalable web environment synthesis for gui agent training. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics*.

Boyuan Zheng, Boyu Gou, Jihyung Kil, Huan Sun, and Yu Su. 2024a. GPT-4V(ision) is a generalist web agent, if grounded. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 61349–61385.

Longtao Zheng, Zhiyuan Huang, Zhenghai Xue, Xinrun Wang, Bo An, and Shuicheng Yan. 2024b. AgentStudio: A toolkit for building general virtual agents. *arXiv preprint arXiv:2403.17918*.

Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. 2023. Webarena: A realistic web environment for building autonomous agents. *arXiv preprint arXiv:2307.13854*.

A Environment Construction and Quality Evaluation

The appendices provide protocols, implementation details, per-site results, and source inventories for the completed experiments.

A.1 Historical Construction and Cost Records

Construction reports for the historical 85-site subset record 59 passed sites, 15 source-blocked sites, 6 failed sites, and 5 sites with missing reports. These statuses reflect different versions of acceptance criteria, visual algorithms, and thresholds and serve to trace historical construction processes. After initial setup and authorization, this subset contains no recorded per-site manual implementation, debugging, or repair. Conclusions about

human intervention and pricing are restricted to this historical subset and the recovered calls.

This construction experiment used the Full Coverage policy. In the recovered SDK records, the Builder and Verifier primarily used `gpt-5.6-sol` with `xhigh` reasoning; some Builder calls used `gpt-6-astra`, so the entire batch did not use a single model.

Recovered model-call and partial cost records cover 84/85 sites. Input tokens, including cached tokens, total 11.207 billion, of which 10.672 billion are cached input tokens; output tokens total 38.4318 million. Recorded, quantifiable model costs total $220.62, with recorded Builder and Independent Verifier costs of approximately $142.52 and $78.11, respectively. Across the 84 sites with records, the mean recorded per-site cost is approximately $2.63 and the median is approximately $1.60. Independently rounded components sum to $220.63, differing from the total by $0.01 due to rounding. Seven sites additionally contain unpriced Astra calls. These costs therefore provide only a lower bound for recovered, priceable model calls. They exclude unpriced calls, browser execution, and other system overhead, are not extrapolated to the total cost of 244 sites, and are not interpreted as the marginal cost of the Verifier under a controlled ablation.

Costs distinguish system development, initial authorization/setup, per-site implementation/debugging/repair, and research auditing. Neither the 244-site artifact pool nor the historical 85-site subset has a complete attempt cohort under uniform acceptance criteria, so we do not compute a unified construction success rate.

A.2 Evidence Scope of Independent Evaluation

The categories, screenshot hashes, and audit statuses of the 244-site inventory are archived with the supplementary materials. Automatic metrics for the five sites and 14 states in RQ2-A derive from per-page scoring files and equally weighted per-site aggregation. The six screenshots displayed in Figure 4 are linked through scoring records and capture hashes; all nine original illustrative screenshots remain archived with the figure's provenance manifest. Historical-reference and mirror screenshots are traced separately from the text extraction used for Text F1; construction-stage checks and the paper's independent measurements are recorded separately.

RQ3 reports paired comparisons on the tasks for which both models obtained valid results and separately reports complete task denominators and infrastructure failures. Evaluation settings, exact counts, and rounding rules appear in Appendix E.1. Results use descriptive statistics.

Table 3 lists the external artifacts actually included. The four additional sites matched between our artifacts and VeriEnv include artifacts imported from existing directories; their execution results do not retroactively count toward independent SDK construction success. Test adapters connect only execution, measurement, and initial-state recovery interfaces; they neither repair nor simplify the implementations under test. Different subsets are reported separately rather than merged into a ranking.

System	B sites	A states and reference limitations
WebHarbor	5/5	Five sites/14 states; historical Apple and ESPN references have rendering defects
VeriEnv	5/5	LinkedIn/four states; Coursera is excluded from historical paired scoring; the other three sites lack historical references

Table 3: External artifact eligibility for RQ2. A denotes historical paired fidelity; B denotes offline interaction validity.

A.3 Historical Source-State Fidelity: References and Sensitivity Analysis

Historical references for Apple and ESPN have rendering defects, and JavaScript rendering conditions differ between source and mirror pages. We retain them in the main results and conduct a sensitivity analysis excluding these two sites: SSIM is 0.8051 for WEBANYTHING and 0.6885 for WebHarbor. This analysis does not replace the five-site main results. Historical paired evaluation of VeriEnv covers four LinkedIn states, with Pixel/SSIM of 0.9574/0.8190 and 0.9436/0.7637, respectively; Text F1 is N/A. Its historical references contain personalized Chinese content, whereas some mirrors contain synthetic English data, limiting case interpretation. Coursera is excluded from historical paired scoring, and the other three sites lack historical references. Current-source spot checks are reported separately in Appendix A.4.

Text F1 in Figure 4 corresponds to the displayed page state rather than the site mean; human labels come from the corresponding frozen blind-audit

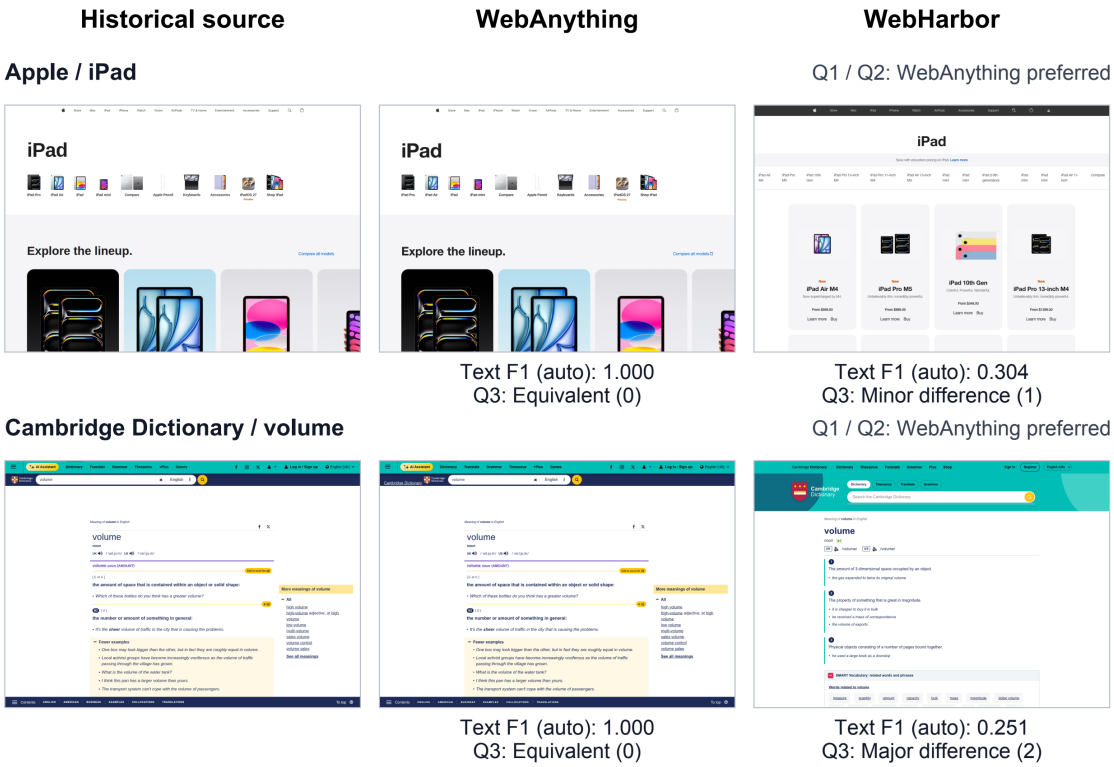


Figure 4: Blind-audit page comparisons between historical source references and reconstructed mirrors. Rows show Apple iPad and a Cambridge Dictionary entry. Q1/Q2 indicate overall and information-fidelity preferences; Q3 indicates discrepancy severity: 0 equivalent, 1 minor, and 2 severe. Automated Text F1 is annotated separately from human ratings. The historical Apple reference records a failed font-stylesheet request. These are selected examples; full blind-audit results for 14 states appear in Table 1a.

samples. Different metrics need not agree: for the arXiv abstract page not displayed in the figure, SSIM is 0.631 for WebAnything and 0.655 for WebHarbor. Its WebHarbor 404 state remains in the original screenshots and complete evaluation statistics.

Historical Text F1 uses full-page `body.innerText`, NFKC and whitespace normalization, and case-sensitive word/punctuation token multisets. Precision and recall use multiset overlap relative to mirror and source token counts. The archived protocol and extraction scripts replay frozen source HTML and resources with JavaScript disabled, retaining historical static normalization; mirror text comes from the earlier JavaScript-enabled captures. Source text is therefore a replay-derived reference, not a contemporaneous live-source text snapshot. This archival rendering asymmetry can affect Text F1 and distinguishes it from matched live rendering; the recorded scores are retained.

Freezing, execution, and denominator rules. The resource inventory and environment-quality evaluation materials in this section use records frozen on October 9, 2026: resource scale comes from the frozen 244-site inventory, historical construction/cost audits from the 85-site subset, and quality results from two independent artifact comparisons. The construction-stage Verifier drives repair and recapture according to site contracts. The paper's independent evaluation harness is separate: it neither reads historical verification conclusions nor modifies the artifacts under test, and instead retests each artifact using common metrics. Artifacts, historical references, and runtime configurations are frozen separately. All failures, 404 responses, source-reference defects, blocked tests, and missing references are retained; no composite environment-quality score is constructed.

Historical-state comparisons use the same designated reference, browser, and viewport, recording limitations in the comparability of page states, languages, and JavaScript rendering conditions. The evaluation harness is independent of construction

methods and implementation stacks. State recovery uses explicit test adapters rather than requiring external artifacts to adopt WEBANYTHING's Flask/SQLite architecture. Interaction tests disable external network access, use a fixed planned set, and record reasons for unexecuted and failed tests. A historical `passed` status cannot substitute for retesting.

A.4 Current-Source Spot Checks

We collected current-source references for five sites and 12 page states frozen in advance. Source access used fresh anonymous sessions with a common 1440×1000 viewport, DPR=1, en-US, and UTC. We waited 15 seconds after DOM loading and disabled screenshot animations at the top of the page. Only read-only requests were permitted on source sites; mirrors used isolated copies with external network access disabled. Pixel is defined as $1 - \mathrm{MAE}/255$, SSIM uses a uniform RGB 7×7 window, and Text F1 measures token overlap in full-page `body.innerText`. The original protocol and per-site results are preserved in the supplementary materials.

Valid source references cover four Coursera and Weather pages. Three render normally on both sides and enter the paired similarity statistics (Table 4). The current Coursera source is the learner homepage; the selected WebAnything-side artifact is an older learner homepage, whereas VeriEnv's root page is Coursera for Campus. Catalog data for Python search also differ. Weather's source homepage is localized to Hong Kong, whereas the selected artifact shows New York weather; weather, news, and advertising all vary over time. These spot checks therefore measure descriptive similarity to the pages at capture time and are reported separately from historical-reference evaluation. The selected Coursera/Weather artifacts come from existing site directories, and conclusions concern these artifacts.

Two pages each on Apartments and Discogs returned 403 responses or access challenges. Four LinkedIn pages returned login or public landing pages that differed from the mirrors' post-login states. These eight states remain classified as source-unavailable. The Weather New York page rendered normally in the selected WebAnything-side artifact, whereas VeriEnv returned 500 offline because of an external API dependency. This runtime failure remains in the planned-state records, and paired similarity means use only the homepage. This check evaluates page presentation and runtime availability; interaction-test results remain those of the independent RQ2-B records.

A.5 Website Stratification and Coverage Statistics

The resource inventory is deduplicated by unique site identifiers and contains 244 artifacts. The first- and second-level usage-category fields correspond to 14 application domains and 53 fine-grained categories, respectively; source hostnames and application domains are recorded separately. Categories retain the initial resource-table annotations, and the pool is not a preregistered random sample with fixed quotas. Site aliases, multiple snapshots, or candidate versions do not count as additional sites. Usage categories in RQ1, tested capabilities in RQ2-B, and actual trajectory visitation scopes are counted separately and are not interchangeable.

A.6 Presentation Diagnostics and Quality Depth of the Resource Pool

All 244 startup records in the frozen inventory contain HTTP 200 responses and homepage screenshots, but these do not represent complete functional acceptance. The 113 additionally reviewed sites comprise 70 without confirmed anomalies, 9 with flagged presentation issues, 26 with signals awaiting diagnosis, and 8 other special records. The remaining 131 sites received no additional review.

SHA-256 and 256-bit difference-hash diagnostics across all frozen screenshots found byte-identical blank homepage screenshots for Cohere and United Airlines. Their historical records correspond to script execution failure and a JavaScript chunk 404, respectively. Fifteen screenshot pairs with difference-hash distance at most 12 are used to locate low-content pages, not to identify duplicate implementations.

The historical Best Buy artifact contains 184 programmatically generated SVGs, and its homepage explicitly uses locally synthesized product and store data. This confirms that some colored blocks are synthetic resources rather than loading failures. Fictional business data do not themselves imply functional invalidity, nor do they establish the pool-wide prevalence of templated implementations. Presentation diagnostics are reported separately from source fidelity and interaction tests; test results for other candidate versions are not reassigned to the current resource pool.

Site	Artifact	Paired/preset states	Pixel	SSIM	Text F1
Coursera	Selected pool artifact	2/2	0.7017	0.4742	0.4236
	VeriEnv	2/2	0.7466	0.5338	0.1928
Weather	Selected pool artifact	1/2	0.6373	0.4386	0.1955
	VeriEnv	1/2	0.5188	0.4018	0.1396
Apartments	Both; source 403	0/2	N/A	N/A	N/A
Discogs	Both; source 403/access challenge	0/2	N/A	N/A	N/A
LinkedIn	Both; pre-login states	0/4	N/A	N/A	N/A

Table 4: Current-source spot checks. Paired/preset states retain all planned states; means use the same pages that render normally on both sides. Source unavailability and mirror runtime failures are reported separately. Product scope, location, and live-content conditions follow the protocol in this section.

A.7 Interaction Tests and State Recovery

Artifact-level tests use fixed site-capability sets covering clicks, text entry, retrieval, navigation, form feedback, and state modification. Capability coverage uses the number of planned capabilities as its denominator; supported and actually tested capabilities count in its numerator even when execution fails. Executed-test success rate uses only executed tests as its denominator, whereas all-planned-test pass rate retains unsupported and blocked items. Image-resource errors are reported separately rather than mixed into interaction-test success rates.

All-planned-test results for the five VeriEnv sites are LinkedIn 6/9 vs. 7/9, Apartments 6/6 vs. 4/6, Coursera 6/6 vs. 4/6, Discogs 6/6 vs. 3/6, and Weather 5/6 vs. 3/6 (selected pool artifacts vs. VeriEnv), totaling 29/33 vs. 21/33.

In the five-site WebHarbor subset, WebHarbor achieves higher coverage and interaction-test completion rates. WEBANYTHING's Cambridge lacks explicit no-results feedback for nonexistent words. WebHarbor's Apple login session becomes invalid after restarting, but shopping-bag state persists. Ten distinct image-resource errors on ESPN are recorded separately.

Initial states are restored through explicit test adapters, and native reset-API support is recorded separately. Persistence/recovery checks are performed only after the corresponding state modification succeeds. VeriEnv's executed persistence checks are 4/4, with two additional planned checks unexecuted; these remain in the planned set. This evaluation reports adapter-level recovery results and does not imply that all artifacts have native reset interfaces.

B System Implementation and Verification Feedback

B.1 Propagation of Evidence, Candidates, and Feedback

The Builder reads existing source materials, candidates, and the latest repair package in the workspace, preserving valid captures and implementations when resuming execution. The independent verification runner reads construction reports and evidence directories and returns verification reports and repair requests. The outer controller records execution statuses and sends failed reports back to the Builder; source blocking and authorization requirements are handled separately. Task generation, interaction collection, and evaluation use frozen environments.

B.2 Version Association

Sites, source snapshots, candidates, and release packages have separate stable identifiers. Version associations for tasks, trajectories, and evaluation results should identify specific implementations, resources, initial data, and runtime configurations rather than relying solely on site names. Training and evaluation protocols and data-overlap checks appear in Appendix E.1.

B.3 Site Contracts and Verification-Report Fields

The inspected contracts associate sites, coverage configurations, viewports, page paths, workflows, and acceptance requirements, with optional interaction contracts and state assertions. Specific checks are determined by the corresponding execution reports.

Independent verification reports record conclusions, candidate digests, page and workflow checks, source limitations, and evidence. Repair packages

System	Capability coverage	Executed-test success rate	All-test pass rate	Startup	Persistence	Initial-state restoration
Five-site WebHarbor comparison						
WEBANYTHING	15/20 (75%)	19/20 (95%)	19/30 (63.3%)	5/5	1/1	1/1
WebHarbor	16/20 (80%)	23/23 (100%)	23/30 (76.7%)	5/5	2/3	3/3
Five-site VeriEnv comparison						
Selected pool artifacts[a]	22/22 (100%)	29/31 (93.5%)	29/33 (87.9%)	5/5	6/6	5/5
VeriEnv	20/22 (90.9%)	21/28 (75.0%)	21/33 (63.6%)	5/5	4/4[b]	5/5

[a] The four added sites include artifacts imported from existing directories; comparisons use the frozen environments. [b] Persistence 4/4 refers to executed checks; two additional planned checks were not executed.

Table 5: Interaction validity of selected frozen environments. WebHarbor and VeriEnv are tested on their respective five-site subsets. We report capability coverage, success rate over executed tests, and pass rate over all predefined tests. External network access is disabled; explicit test adapters restore state.

associate issue locations, expected and observed outcomes, corresponding evidence, and repair suggestions. The model determines diagnostic actions using the reports and existing source evidence.

B.4 Construction-Stage Verifier Implementation and Versions

The construction-stage Verifier uses site contracts, model-executed browser tests, and site-specific assertions. Visual comparison may combine screenshots, DOM, element geometry, and auxiliary scripts. Historical versions did not use a unified SSIM, LPIPS, or L1–L4 composite score. Fidelity v19 introduced normalized RGB pixel similarity, but algorithms and thresholds still differ across sites.

Reset prechecks include programmatic hard checks dependent on WEBANYTHING's Flask/SQLite architecture; they are not stack-independent external-baseline scorers. Earlier runners did not enforce positive- and negative-case calibration acceptance criteria for evaluators. Task-evaluation calibration therefore requires independent evidence.

Reports preserve concrete check evidence, with top-level conclusions distinguishing passed, failed, and source-blocked outcomes. The outer runner consumes reports rather than recomputing a unified four-layer score. The historical artifact pool contains candidates from different versions with passed, failed, source-blocked, and missing-report statuses. The independent evaluation harness retests frozen artifacts without reading these historical verification conclusions.

B.5 Resumption Prompt for the Explorer and Builder

The Builder Agent directly receives a real website URL and manages exploration, evidence collection, and local reconstruction within the same context. The historical resumption template requires preserving existing source evidence, candidate implementations, and repair records, reading the latest check feedback, and continuing repairs from failed or unfinished items. It returns to the source for targeted recapture only when existing evidence cannot answer a concrete question. Its final submission comprises the candidate implementation, local resources, and a self-check report containing actual check results and limitations. This template resumes checkpoints after timeouts; it is not the sole initial prompt shared by all construction batches.

The complete historical English template appears in the separate prompt supplement; this description does not replace the original instructions.

B.6 Independent Final Verifier Prompt

The Independent Verifier inspects frozen candidates in a separate context without modifying the Builder's candidate code. It performs actual browser checks based on source evidence and site contracts, covering applicable pages, controls, resources, state persistence, and recovery, and produces feedback containing concrete failure locations, expected outcomes, observed outcomes, and evidence. Stateful checks modify only local runtime states and restore initial states before finishing. Candidates modified by the Builder must be frozen again and independently reverified.

The English template and unexpanded input/output variables are preserved in `supplementary-prompts.tex`. Internal historical names are anonymized as

Independent outcome	Pairs	Share
Passed	56	25.57%
Failed	**139**	**63.47%**
Source blocked	24	10.96%
Of these: evidence-supported self-check omissions	**39**	**17.81%**

All shares use 219 builder-pass pairs as the denominator; the 39 pairs span 22 sites. Candidate identity, check scope, or runtime conditions are not strictly matched in every pair, so not all 139 failures establish self-check omissions. Only one check pair strictly matches both candidate and semantic target.

Table 6: Independent verification of submitted candidates with passing builder self-checks. The first three outcomes are mutually exclusive; the last row is a subset of failures supported by concrete defect evidence.

construction-system identifiers, retaining the original instructions and system attribution. The template's construction-acceptance viewport is 1920×992, distinct from the 1440×1000 viewport used for independent quality evaluation. Executed historical checks are documented in the corresponding records.

B.7 Evidence Definitions for Verification Decoupling and Feedback Analysis

The analysis uses archived construction, self-check, independent-verification, and repair records without rerunning construction or checks. The 229 pairs retain actual associations between construction and verification reports. Builder-pass/Verifier-fail status discrepancies occur in 139 pairs, of which 39 have supporting evidence of concrete check targets and defects. Supporting evidence includes one strictly candidate-matched subcheck and 38 non-strict pairs; sites can overlap across evidence levels.

Among all pairs, three pairs across two sites have identical observed hashes and explicitly identical algorithms, but only one resource subcheck for Stashmedia also satisfies the same semantic target and actual-defect criteria. Other pairs may differ in candidate identity, coverage, protocol, or access conditions. The absence of a target status discrepancy does not prove self-check completeness. Report-level repair chains across 46 sites follow existing historical records and are defined by traceable feedback reading, modification, new digests, and subsequent reverification; they do not count resolution of all initial issues.

The issue table contains 1,007 records, of which 49 received item-level semantic adjudication. The 18 confirmed records jointly demonstrate feedback execution, candidate modification, and a corresponding semantic-reverification endpoint. A later overall pass alone does not establish issue closure. Remaining unadjudicated or incomparable records cannot be treated as repair failures. The 18 records include shared mechanisms, repeated manifestations, and recurrences and are not treated as independent bugs or independent statistical samples. Existing records support actual recapture on two sites, but do not establish closure of the original evidence gaps targeted by recapture under comparable checks; we therefore do not report causal effects of recapture.

B.8 Evidence for Verification and Feedback Cases

Stashmedia. The Builder reports no external requests or broken images in resource checks across 25 routes. The Independent Verifier identifies external image requests, broken images, and console errors on three archive routes. Both sides use the same candidate-digest algorithm, with matching digests and file counts (626), supporting a same-candidate resource-check comparison. The conclusion is restricted to this subcheck; identical waiting and browser-execution conditions have not been fully established.

Ctrip. Repair requests record five issues: invalid login, destination queries, favorites, booking drafts, and reset. Construction event streams confirm that feedback was read and that application and runtime-control implementations were subsequently modified. The updated candidate has corresponding semantic-reverification pass records in independent workflow and initial-state recovery checks. The initial failures of favorites, drafts, and reset involve a read-only filesystem/SQLite, so these three records are classified as runtime-environment repairs, and the first two as local-implementation repairs. The initial failing round is outside the strictly associated 229-pair cohort and is reported separately as a repair case, not counted among the 22 sites with omissions in that cohort.

Figma. An authorized source capture established a canvas-anchored comment composer. Verification feedback identified mismatched placement and appearance. The event stream records feedback consumption and changes to the editor markup, styles, and interaction code. Independent targeted checks on the updated candidate confirmed the

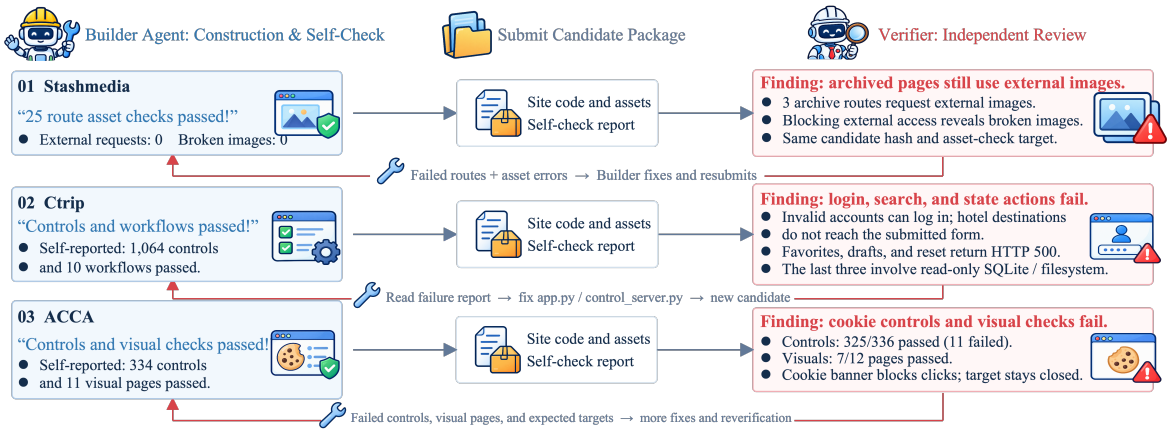


Figure 5: Historical verification cases. Stashmedia reveals a same-candidate resource-check omission; Ctrip traces feedback-driven revision and issue-level independent reverification; ACCA illustrates additional control and visual defects. Evidence scopes are specified in Appendix B.8.

composer's geometry and visibility. This example illustrates source-supported Local Repair, without attributing resolution to Targeted Recapture.

C Task Supply and Data Statistics

C.1 Interfaces for Downstream Task Generation, Evaluation, and Collection

Task and training-data construction use released environments. This section describes the implementation interfaces for task specifications, runtime recovery, evaluation, and data binding.

From environment capabilities to tasks. The task pipeline reads the pages, visible objects, native operations, and seed data of a specific mirror to formulate goals such as retrieval, filtering, detail verification, comparison and aggregation, and state modification. Existing planning materials record whether the required facts reside in lists, detail pages, or other pages, and organize applicable operations into task specifications. Natural-language instructions express user goals, while answers, evidence supporting the operations, and evaluation materials remain hidden. The executable scope of a task is bounded by the capabilities and initial state actually supported by the environment.

Initial states and success conditions. Public task records contain task identifiers, instructions, sites, starting URLs, and login and reset requirements. Task specifications record capability tags, action and step budgets, and successful and unsuccessful terminal states. Hidden specifications associate required objects, query parameters, source evidence, expected results, and output contracts. Success conditions for information tasks may specify fields, sets, orderings, or comparison results in normalized answers. State-changing tasks check application states after saving, creating, modifying, or restoring objects; tasks admitting multiple solutions use sets of state predicates. When necessary, fictional test records are prepared in private copies of native data structures, while original environment assets remain read-only. Task answers are grounded in the task's initial data. Task text or a nonempty answer file alone does not establish evaluability: corresponding executable checks must exist.

Evaluator construction and execution-based verification. During generation, site-specific scripts first check native routes, evidence visibility, and applicable answer, state, and reset conditions. A visual agent then receives instructions and observations in the initial state of the corresponding mirror through the fixed NorthCUA harness and performs browser operations. An independent task evaluator checks the final answer against hidden success conditions and determines goal completion using the terminal DOM, screenshots, and applicable state evidence. Model-based evaluation is used for applicable semantic conditions and is separated from agent execution. Tasks with evaluation materials become end-to-end accepted tasks only after actual interaction and outcome checks, rather than receiving a pass solely because specifications exist.

Recovery, binding, and training entry points. Task execution packages locate native implementations, immutable seeds, and starting routes through runtime descriptions. When login is required, they restore fictional seed identities while isolating real user sessions. Before task execution, mutable databases are restored from seeds, and site-specific reset and startup checks verify applicable states. Tasks whose runtime prerequisites are not met are recorded as blocked. Task identifiers, source-data versions, and specification digests connect public tasks to hidden evaluation materials. Collection packages record implementations, resources, initial data, runtime configurations, and archive digests to associate the complete environment identity.

The pipeline separately records task generation, evaluator construction, execution-based acceptance, trajectory collection, and training-sample export. Saved successful trajectories are used for training after quality filtering, format conversion, and step-level sample export. The 6,172 accepted tasks in the earlier source collection, the 5,856 trajectories used for SFT, the 5,167 unique training task IDs obtained by deduplicating task IDs in the training manifest, and the 60,517 step-level samples are counted in their respective units. The relationship between the separate batch of 6,812 new tasks across 229 sites and the 74 source sites follows the set definitions in the experimental setup.

Task attribute	Tasks	Share	Sites
Information retrieval	4,802	77.8%	74
Filtering and search	2,581	41.8%	72
Comparison and aggregation	1,549	25.1%	68
Multi-page tasks	6,157	99.8%	74
State operations	1,655	26.8%	68
Cross-site tasks	2,326	37.7%	28

Table 7: Automatically assigned multi-label capability statistics for 6,172 accepted tasks.

C.2 Diversity and Interaction Complexity of Accepted Tasks

This section reports statistics for the 6,172 accepted tasks in the post-training source collection, covering 74 site labels. Agents executed these tasks in the environments, and task evaluators confirmed success. This collection and the separate collection of 6,812 newly generated tasks across 229 sites have no task-level containment relationship; the 74 source sites are a subset of the 229 sites. The 6,172 source tasks, 5,856 training trajectories, 5,167 unique training task IDs obtained by deduplicating task IDs in the training manifest, and 60,517 step-level samples are distinct counting units. Overlapping site names do not establish task-level or environment-version correspondence.

Capability attributes. The labels in Table 7 are automatically generated from instructions, execution trajectories, and website information. They may overlap and are not human gold-standard annotations. Task counts are computed per attribute, and site counts reflect the sites associated with that attribute. Neither column can be summed across rows to obtain a total number of unique objects. The multi-page attribute retains its automatic-label definition and is not directly equivalent to the number of unique paths after query parameters are removed.

Observed interaction complexity. Table 8 uses actual execution trajectories rather than planning estimates. Tool-interaction steps count tool operations during execution; URL page states retain query parameters, whereas page paths discard them. These three statistics are not interchangeable. Other URL-normalization and repeated-state handling rules must accompany the statistical scripts; we do not infer unreported deduplication rules from aggregates. Step-count bins appear in Table 9: 2,139/6,172 tasks exceed 10 steps (approximately 34.7%), and 641/6,172 exceed 20 steps (approximately 10.4%). These values demonstrate variation in interaction depth, not that all tasks are long-horizon. Zero-step tasks remain in the total denominator and are not excluded post hoc.

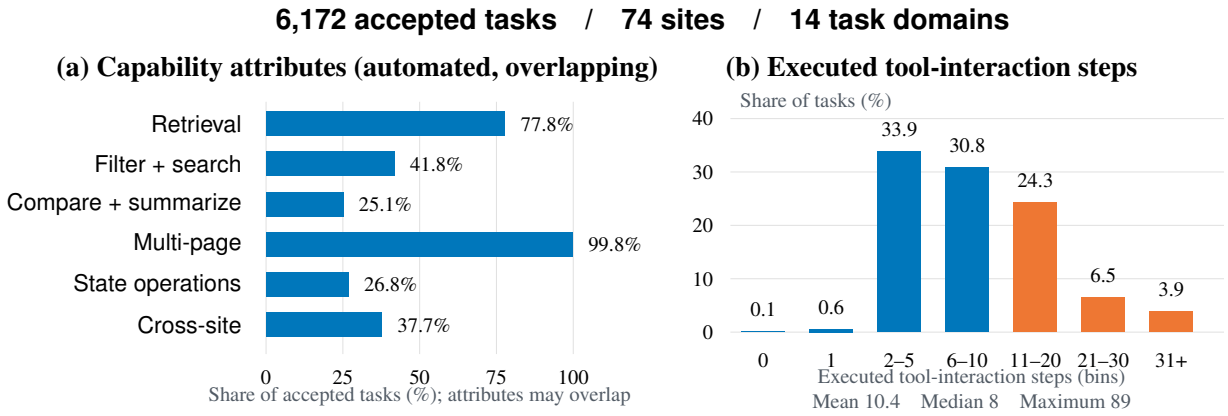


Figure 6: Diversity and interaction complexity of 6,172 accepted source tasks spanning 74 site labels and 14 task application domains. (a) Six automated, overlapping capability attributes derived from instructions, trajectories, and website information, each using 6,172 as the denominator. (b) Grouped distribution of executed tool-interaction steps; approximately 34.7% of tasks exceed 10 steps.

Metric	Mean	Median	P75	P90	P95	Maximum
Observed tool-interaction steps	10.4	8	13	21	28	89
Observed URL page states	5.5	5	7	9	12	63
Page paths excluding query parameters	4.7	4	6	8	10	62

Table 8: Observed interaction complexity of accepted tasks. Tool operations, URL states, and page paths are counted separately from actual trajectories.

Tool-interaction steps	Tasks	Share
0	4	0.1%
1	38	0.6%
2–5	2,092	33.9%
6–10	1,899	30.8%
11–20	1,498	24.3%
21–30	399	6.5%
31 or more	242	3.9%
Total	6,172	—

Table 9: Bins of observed tool-interaction steps. Rounded shares may not sum to 100%.

C.3 New Task Batch and Acceptance Definitions

The new batch and the earlier collection of 6,172 post-training source tasks are independent sets. The earlier 74 source sites are a subset of the 229 sites in the new batch. Within the 244-site artifact pool, 229 sites generated 6,812 tasks that completed visual-agent execution and corresponding task-evaluator acceptance. Of these, 217 sites reached 30 tasks each, 12 generated 1–29 tasks, and 15 have no generated tasks. Across all 244 sites, the median is 30 tasks per site, the range is 0–30, and the mean is 27.9. Target-quota completion is 6,812/7,320 (93.1%). These counts describe accepted outputs, not the success rate of all proposals or execution attempts.

In the historical-materials snapshot of October 10, 2026, public tasks and metadata each contain 6,812 records, with unique task identifiers and matching sets. Hidden-criterion identifiers match 6,782 records, and runtime-description files exist for 6,540 records. This measures association completeness in archived materials, not execution pass rate. Complete version correspondence between this snapshot and end-to-end acceptance records has not been established; differences are not counted as execution failures. Environment identity also requires associating specific implementations, resources, initial data, and runtime configurations.

Task generation, evaluation materials, script checks, end-to-end acceptance, trajectory collection, and training export are counted separately. The SFT collection is specified in Section 4.4; trajectory-length statistics use observed actions.

D Human Blind Audit: Protocol and Results

The human blind audit evaluates historical source-state fidelity in RQ2-A. All five sites and 14 page states in the WebHarbor-matched historical-state subset are included: Apple, arXiv, Cambridge Dictionary, ESPN, and Hugging Face Papers. There is no sampling or selection based on automatic scores. Each sample simultaneously displays the historical source reference, anonymous mirror A, and anonymous mirror B, with A/B order randomized per sample. Three annotators participate, forming final labels through discussion. Annotators do not know system identities or see Pixel, SSIM, or Text F1.

Rating questions. Q1 (overall source-state fidelity) asks which mirror is closer overall to the designated historical source page, with choices A better/tie/B better. Q2 (information fidelity) asks which mirror more completely preserves visible information that may affect agent observation or decisions, using the same choices. Q3 (agent-relevant discrepancy severity) separately rates each mirror against the historical source reference: 0 denotes equivalent, 1 minor differences that do not affect functionality, and 2 severe differences that may affect agents; lower is better. The target is always the designated historical source state. Annotators are not asked to infer functional correctness for operations that were not executed.

Consensus annotations. All wins/ties/losses are computed for WEBANYTHING relative to WebHarbor. Q1 records 12 wins, 2 ties, and 0 losses, with a win rate of $12/14 = 85.7\%$. Q2 records 10 wins, 3 ties, and 1 loss, with a win rate of $10/14 = 71.4\%$. Both rates use all 14 pages as the denominator; ties are not removed. The sole WebHarbor win in Q2 corresponds to ESPN.

For Q3, WEBANYTHING's equivalent/minor/severe distribution is 12/14, 2/14, and 0/14, with mean severity $(0 \times 12 + 1 \times 2 + 2 \times 0)/14 = 0.143$ (rounded to three decimals). WebHarbor's distribution is 0/14, 8/14, and 6/14, with mean severity $(0 \times 0 + 1 \times 8 + 2 \times 6)/14 = 1.429$. Severe-discrepancy shares are 0% and 42.9%, respectively. These quantities are reported separately, not combined into a new overall fidelity score.

Labels and interpretation scope. We report consensus labels formed through discussion among three annotators and do not use them to compute independent inter-annotator agreement. Human judgments measure visible differences in these 14 designated historical-reference states; limitations arising from source versions, Apple/ESPN reference-rendering defects, and JavaScript rendering conditions remain applicable. Interaction validity is measured in RQ2-B; the reliability of Verifier conclusions is a separate calibration objective.

E Training and Evaluation Configuration

E.1 RQ3: Protocol for Post-Training Utility

We compare the Qwen3.5-9B Instruct baseline with the model supervised-fine-tuned on environment

interaction data, using the step-473 checkpoint at the fixed training endpoint.

Training inputs and recipe. The task configuration actually reads the non-thinking input file, whose independently counted line total is 60,517. The merged data manifest records 5,856 trajectories: 4,651 from the historical batch and 1,205 from the incremental batch. Deduplication by task ID in the training manifest yields 5,167 unique training task IDs across 74 site labels. Versions and format variants of the same task are retained, so trajectory counts are not counts of semantically unique tasks. Filtering requires task-evaluation success and a mean score of at least 4 across process, completion, and efficiency; it does not require each dimension to be at least 4. Only the final agent turn in each sample is supervised, and image coordinates are normalized to 1000.

The nominal number of data passes follows from $473 \times 256/60{,}517 \approx 2.001$. It describes the configured budget and does not replace per-sample observed exposure counts. SHA-256 digests are preserved separately for the actual input file, merged trajectory manifest, launch configuration, and evaluation configuration; model usage and effective training tokens require separate log statistics.

Provenance and overlap checks. All training data come from WEBANYTHING's historical interaction-collection batches. Exact matching after lowercasing and word normalization across 5,856 training instructions, 102 REAL tasks, and 200 Odysseys tasks found no identical text. For test instructions with at least eight words, screening for word-set Jaccard similarity of at least 0.65 also found no candidates (100 REAL tasks and 200 Odysseys tasks). These checks do not cover semantic paraphrases, shared task templates, or test-derived relationships.

Of the 74 training site labels, 69 map to source hosts through the current resource table. Comparing evaluation entry points against literal hostnames after removing the `www.` prefix found no matching host overlap for REAL. The intersection of all Odysseys entry points with training sources includes Amazon, Apple, Google, and YouTube. This check does not cover every host visited during task execution, nor does it map REAL application aliases to unconfirmed original-site domains. Source-host overlap does not imply use of the same mirror version, and different entry points do not guarantee cross-site isolation. This comparison uses the step-473 checkpoint at the training endpoint.

REAL. The full set contains 102 tasks, with 11 infrastructure failures for the post-trained model. Main results use the 91 tasks with valid evaluation results for both models: the baseline succeeds on 22/91 (24.2%) and the post-trained model on 31/91 (34.1%), a net gain of nine tasks (+9.9pp). Per-task paired outcomes comprise 15 failure→success transitions, six success→failure transitions, and 70 unchanged outcomes, totaling 91. The complete-denominator robustness analysis retains all 102 tasks and does not count the 11 infrastructure failures as successes: 22/102 (21.6%)→31/102 (30.4%), +8.8pp. The two denominators are reported separately, and per-task changes use descriptive counts.

Odysseys-Easy. Both models use the same NorthCUA hybrid harness, with `gpt-5.6-sol` as the judge and `max_images=40`. The model-execution budget is at most 100 NorthCUA model turns and 5,400 seconds per task. REAL uses 30 model turns and 10,800 seconds per task, with matching task-selection digests for both models. These time budgets come from the corresponding runtime configurations; execution retries and concurrency are preserved separately in runtime records. The baseline completes 45/45 tasks; the post-trained model has 44 valid results and one infrastructure failure. The primary comparison uses the 44 tasks with valid results for both models. Baseline and post-trained Rubric Avg are 19.71% and 25.96% (+6.25pp); Task-average is 18.86% and 25.18% (+6.32pp); Perfect is 2/44 (4.55%) and 4/44 (9.09%). The Perfect gain is computed from raw counts: $100(4/44 - 2/44)$ is approximately +4.55pp.

The baseline's full 45-task reference is reported separately as Rubric Avg 21.13%, Task-average 20.22%, and Perfect 2/45 (4.44%); it does not replace the baseline on the 44 mutually valid tasks. The post-trained model's 45th task remains an infrastructure failure. The 44 tasks contain 208 rubric conditions. Rubric Avg is a condition-level micro-average, with baseline and post-trained counts of 41/208 and 54/208. Task-average is the macro-average of per-task rubric means, and Perfect requires every condition of a task to be satisfied. Aggregates are recomputed from each model's saved results without mixing records from other judges. Results cover the Easy subset of Odysseys.

Item	Recovered configuration
Training method and parameter scope	Supervised fine-tuning; trainable language model, frozen vision encoder and projection layer; BF16
Training-data counts	6,172 accepted source tasks; 5,856 training trajectories; 5,167 unique training task IDs; 60,517 step-level samples
Samples and checkpoint	60,517 step-level samples; step-473 checkpoint; approximately two nominal data passes
Optimizer	Adam with decoupled weight decay 10^{-4}; $\beta_1 = 0.9$, $\beta_2 = 0.95$, $\epsilon = 10^{-8}$
Learning rate and schedule	5×10^{-6}, minimum 0; cosine decay; 24 warmup steps
Batch size and randomness	Global batch size 256, microbatch size 1; random and mixing seeds both 42
Sequence handling	Maximum 262,144; no packing, no forced padding to the maximum, truncation for over-length sequences
Image configuration	Minimum 65,536 pixels, maximum 16,777,216 pixels; non-thinking processor
Parallelism and hardware	8 nodes with 8 GPUs each, 64 GPUs total; tensor parallelism 2; GPU model not recovered

Table 10: Actual configuration associated with the step-473 training run, cross-checked against command overrides, optimizer logs, and checkpoint records.

WebArena-Verified. We evaluate the same trained checkpoint used for REAL and Odysseys-Easy with the NorthCUA harness. Within WebArena-Verified, the baseline and SFT models use matched task sets, execution budgets, and evaluation settings. Table 2 retains the benchmark's strict scores. Supplementary rescoring of the same SFT outputs ignores JSON-format judgment: Retrieve passes increase from 39/325 (12.00%) to 105/325 (32.31%), a difference of 66 passes (+20.31pp). This is a scoring-protocol comparison without new agent execution. It does not replace the strict comparison or establish the cause of the entire strict Retrieve decline; no format-relaxed baseline or Overall result is reported.

Experimental reference and task distribution. The baseline without additional training provides a model reference from the same starting point; this comparison measures transfer utility after post-training on environment data. Benchmarks are reported independently. Training data are generated through a unified task–evaluator pipeline, with limited coverage of long-horizon, multistage tasks and tasks with strong intermediate-state dependencies. This limitation concerns data-construction capabilities; source fidelity is measured separately in RQ2.

F Reproducibility Materials and Data Governance

Supplementary materials include the 244-site inventory, the 14-domain/53-category mapping, additional-review markers for 113 sites, and original 1440×1000 screenshots and digests. The separate atlas retains all frozen homepages by application domain without filtering presentation or resource anomalies. Historical qualitative comparisons additionally preserve nine original screenshots and their corresponding per-page scores.

For reproduction, the resource pool, historical 85-site subset, RQ2 comparison artifacts, and training sources are associated separately. Models, harnesses, tasks and evaluators, runtime budgets, initial states, test adapters, and reasons for nonexecution are preserved. The evidence scope of each result appears in Appendix A.2.

Environment distribution is constrained by content licenses, privacy, and access authorization; restricted resources are handled separately from publicly shareable anonymized statistics. Side-effecting operations involving payments, posting, deletion, or real-account modification are prohibited by default. Released environments contain neither real credentials nor private user content. AI tools assisted textual organization and formatting of this manuscript; the authors are responsible for verifying experimental numbers and technical conclusions.

F.1 Complete Site Inventory

The following lists all 244 unique site artifacts frozen as of October 9, 2026, grouped by first-level application domain in the resource inventory. Parentheses indicate site counts per domain; there are 14 domains and 53 fine-grained categories. Names and categories follow the frozen inventory. The list describes resource-pool inclusion, not uniform functional acceptance of every site. Artifacts with presentation anomalies are retained; diagnostic definitions appear in Appendix A.6.

E-commerce and retail (33 sites). Amazon; Apple; Backcountry; Best Buy; B&H Photo; Birkenstock; CarMax; Cars.com; Carvana; Dillard's; DJI; Google Shopping; IKEA; JCPenney; Kelley Blue Book; Macy's Wine Shop; Michaels; Micro Center; Parkers; Porsche; Prestashop; Samsung;

SHEIN; Shopify; Speedo; Target; Trader Joe's; UNIQLO; US Appliance; Versus; Warhammer; Wine Access; Zara.

Development and research (32 sites). Anthropic; ArXiv; Choosealicense; Claude Code; Cohere; DBLP; DeepSeek; DJICDN; ElevenLabs; Gitea; GitHub; Gitlab; Google DeepMind; Grafana; Groq; Home Assistant; Hugging Face; ICLR; CERN WWW Project; Kaggle; Microsoft Azure; Mistral; ModelScope; Tencent Cloud; NVIDIA; Openreview; Phys.org; SourceForge; Statista; Superset; Wandb; Zotero.

Travel and local services (30 sites). Airbnb; Amtrak; Apartments.com; Booking; Carnival Cruise; China Eastern Airlines; Compass; China Southern Airlines; Ctrip; Delta; Doordash; FlightAware; Google Flights; LandWatch; Marriott; Megabus; MTA; Opentable; Public Storage; Qatar Airways; Raising Cane's; RE/MAX; Ryanair; SpotHero; Student.com; Thumbtack; TourRadar; Trip.com; United Airlines; Wanderlog.

Finance and enterprise services (26 sites). ACCA; Amazon Jobs; American Express; Better Business Bureau; Cboe; Chase; Chronicle Jobs; Chinese Institute of Certified Public Accountants; CoinMarketCap; Erpnext; Google Finance; Hubspot; IRS Refund; NYSE; OhioMeansJobs; Robinhood; U.S. Securities and Exchange Commission; Shui5; SmartAsset; Super Lawyers; Turbotax; Walmart Careers; Y Combinator; Yahoo Finance; Zendesk; ZipRecruiter.

Entertainment, sports, and media (26 sites). Bandcamp; BBC News; Bilibili; BoardGameGeek; Chess.com; Discogs; Disney; ESPN; Eventbrite; Fox Sports; IGN; IMDb; Jellyfin; League of Legends; NBA; NFL; Playblackdesert; Red Bull; Rotten Tomatoes; SoundCloud; Stashmedia; Steam; StubHub; Ticketek; Ticketmaster; YouTube.

Government and public services (17 sites). ACRA Singapore; National Audit Office of the People's Republic of China; California.gov; Canada Revenue Agency; Data.gov; GOV.UK; Library of Congress; Medicare.gov; NPS.gov Homepage (U.S. National Park Service); Ohio.gov; Recreation.gov; U.S. Customs and Border Protection; U.S. Department of Justice; USA.gov; USCIS; United States DOGE Service; Virginia DMV.

Search, knowledge, and general-purpose tools (15 sites). AccuWeather; Cambridge Dictionary; Drupal; Ghost; Google Maps; Google Search; Huiji Wiki; Merriam-Webster; Tampermonkey; The Weather Network; Weather.com; Wikipedia; Windy; Wolfram Alpha; Wordpress.

Collaboration and productivity (13 sites). Airtable; Asana; Canva; Dropbox; Excalidraw; Figma; Linear; Nextcloud; Notion; Penpot; Surveymonkey; Trello; WPS.

Education (13 sites). UC Berkeley; Civil Aviation Flight University of China; Civil Aviation University of China CUCAS; Coolmath4Kids; Coursera; Instructure; Moodle; Ohio State University; PhET Simulations; Shanghai Innovation Institute; Stanford University; TED; University of Michigan.

Communication and office tools (11 sites). 4shared; Discord; Feishu; Gmail; Google Calendar; Jitsi; Mattermost; MEGA; Rocket Chat; Slack; Verizon.

Social networking and communities (11 sites). 9GAG; Craigslist; Discourse; Fandom; Imgur; Linkedin; Mastodon; Reddit; Threads; Tumblr; X (Twitter).

Healthcare (9 sites). America's Health Rankings; BabyCenter; CVS; Drugs.com; Healthgrades; Healthline; Mayo Clinic; WebMD; WebMD Doctor.

Everyday knowledge (5 sites). Adopt-a-Pet; American Kennel Club; Allrecipes; Cookpad; Petfinder.

Logistics services (3 sites). FedEx; UPS; USPS.